

Architecture Document

RealWorld Spring Boot application (RealWorldApplication)

Table of Contents

- [1. Architecture Overview](#)
- [2. Technical Summary](#)
- [3. Functional Scope](#)
- [4. Application Structure and Layer Responsibilities](#)
- [5. API and Entry Points](#)
- [6. Domain and Data Model](#)
- [7. Persistence and Data Access](#)
- [8. Security Architecture](#)
- [9. External Integrations and Communication Points](#)
- [10. Configuration and Runtime Considerations](#)
- [11. Observability, Testing and Delivery Evidence](#)
- [12. Architectural Patterns and Implementation Characteristics](#)
- [13. Risks, Constraints and Technical Notes](#)
- [14. Recommendations](#)
- [15. Conclusion](#)

1. Architecture Overview

This project is a Spring Boot application implemented in Java (sourceCompatibility 11) with a layered server-side architecture. The application exposes REST HTTP endpoints and includes configuration and code supporting:

- Controller layer (REST controllers under io.spring.api).
- Application / service layer (io.spring.application).
- Core domain interfaces (io.spring.core.*).
- Infrastructure layer using MyBatis mappers and repository implementations (io.spring.infrastructure.*).
- Programmatic security using Spring Security with a JWT filter and stateless session management.
- MyBatis integration and transaction management enabled via Spring.
- Custom Jackson module for Joda-Time DateTime serialization.

2. Technical Summary

Aspect	Details
Language	Java 11 (source/target compatibility set to 11 in build.gradle)
Framework	Spring Boot (plugin version 2.6.3 present in build.gradle)
Build System	Gradle (build.gradle present; test task configured to use JUnit Platform)
Persistence	MyBatis (mybatis-spring-boot-starter dependency; mapper locations configured: mapper/*.xml)
	SQLite (spring.datasource.url=jdbc:sqlite:dev.db in application.

Database	properties)
Security	Spring Security with a JwtTokenFilter bean and BCrypt PasswordEncoder; stateless sessions configured in WebSecurityConfig
API Documentation	OpenAPI / Swagger annotations used (io.swagger.v3.oas.annotations present in controllers)
GraphQL	Netflix DGS GraphQL starter dependency present and /graphql & /graphql paths allowed in security config
Testing	JUnit 5, Spring Boot test, MockMvc and Mockito evidence; ~23 test classes detected
CI/CD	GitHub Actions workflow (.github/workflows/gradle.yml) running Gradle clean test
Containerization	No Docker-related files were found in the repository
Observability	Package-specific logging levels configured in application.properties

3. Functional Scope

Implemented and evidenced functionality (based on controllers, services and detected endpoints):

- User registration and authentication:
 - POST /users — create user (UsersApi).
 - POST /users/login — login and token generation (UsersApi).
 - GET /user, PUT /user — current user endpoints (CurrentUserApi referenced in detected endpoints).
- Article management:
 - GET /articles — list articles (ArticlesApi).
 - POST /articles — create article (ArticlesApi).
 - GET /articles/{slug} — retrieve article by slug (ArticleApi).
 - PUT /articles/{slug} — update article (ArticleApi) with authorization checks.
 - DELETE /articles/{slug} — delete article (ArticleApi) with authorization checks.
 - GET /articles/feed — authenticated user's feed (ArticlesApi).
- Article favorites:
 - POST /articles/{slug}/favorite — favorite an article (ArticleFavoriteApi).
 - DELETE /articles/{slug}/favorite — unfavorite an article (ArticleFavoriteApi).
- Comments:
 - POST /articles/{slug}/comments — add comment (CommentsApi).
 - GET /articles/{slug}/comments — list comments (CommentsApi).
 - DELETE /articles/{slug}/comments/{id} — delete comment (CommentsApi).
- Profiles and follows:
 - GET /profiles/{username} — profile retrieval (ProfileApi).
 - POST /profiles/{username}/follow — follow user (ProfileApi).
 - DELETE /profiles/{username}/follow — unfollow user (ProfileApi).
- Tags:
 - GET /tags — retrieving tags (TagsApi).

All above endpoints are explicitly detected in code or the detected endpoints list.

4. Application Structure and Layer Responsibilities

High-level package responsibilities (based on package names and class evidence):

- io.spring (root)
 - RealWorldApplication.java — main Spring Boot application entrypoint; @SpringBootApplication.
 - JacksonCustomizations.java — Jackson Module bean registering a custom DateTime serializer.
 - MyBatisConfig.java — enables transaction management.
- io.spring.api — HTTP REST controllers and API-layer concerns.
 - Controllers handle request mapping, validation annotations, response shaping, and delegate to application services and repositories.
 - Exception handling under io.spring.api.exception with a global @RestControllerAdvice (CustomizeExceptionHandler).
 - Security configuration under io.spring.api.security (WebSecurityConfig, JwtTokenFilter bean).
- io.spring.application — application services and query services (e.g., ArticleQueryService, ArticleCommandService) implementing business logic orchestration and DTO assembly (ArticleData, UserData references).
- io.spring.core — domain interfaces and repositories (ArticleRepository, CommentRepository, ArticleFavoriteRepository, UserRepository) and domain types referenced (Article, Comment, User, ArticleFavorite).
- io.spring.infrastructure — concrete repository implementations using MyBatis (MyBatisArticleRepository, MyBatisCommentRepository, MyBatisArticleFavoriteRepository, MyBatisUserRepository partial evidence) and MyBatis read services under infrastructure.mybatis.readservice.

Layer responsibilities summary:

- Presentation/API: validate input, perform authentication/authorization checks where applicable, format responses.
- Application/Service: coordinates reads/writes, enriches data for responses (ArticleQueryService has methods to populate favorites counts, following flags).
- Core domain: domain models and repository interfaces.
- Infrastructure: MyBatis mappers and implementations that interact with the relational database.

5. API and Entry Points

Detected REST controllers and endpoints are summarized below.

Endpoint	Controller
GET /articles/{slug}	io.spring.api.ArticleApi
PUT /articles/{slug}	io.spring.api.ArticleApi
DELETE /articles/{slug}	io.spring.api.ArticleApi
POST /articles/{slug}/favorite	io.spring.api.ArticleFavoriteApi
DELETE /articles/{slug}/favorite	io.spring.api.ArticleFavoriteApi
POST /articles	io.spring.api.ArticlesApi
GET /articles/feed	io.spring.api.ArticlesApi
GET /articles	io.spring.api.ArticlesApi
POST /articles/{slug}/comments	io.spring.api.CommentsApi

GET /articles/{slug}/comments	io.spring.api.CommentsApi
DELETE /articles/{slug}/comments/{id}	io.spring.api.CommentsApi
GET /user	io.spring.api.CurrentUserApi
PUT /user	io.spring.api.CurrentUserApi
GET /profiles/{username}	io.spring.api.ProfileApi
POST /profiles/{username}/follow	io.spring.api.ProfileApi
DELETE /profiles/{username}/follow	io.spring.api.ProfileApi
GET /tags	io.spring.api.TagsApi
POST /users	io.spring.api.UsersApi
POST /users/login	io.spring.api.UsersApi

Controllers use OpenAPI annotations (io.swagger.v3.oas.annotations) to document operations and responses.

6. Domain and Data Model

Evidence about domain and data model:

- Domain types referenced in code (source files for some domain types not included in analyzed excerpts):
 - Article (io.spring.core.article.Article)
 - Comment (io.spring.core.comment.Comment)
 - ArticleFavorite (io.spring.core.favorite.ArticleFavorite)
 - User (io.spring.core.user.User)
 - Tag (io.spring.core.article.Tag)
 - FollowRelation (io.spring.core.user.FollowRelation)
- Transport / view-model types referenced by application and API layers (source files not present in the provided materials or not detected):
 - ArticleData, ArticleDataList, ArticleFavoriteCount, UserData, UserWithToken (referenced by services and controllers).
- No evidence of JPA/Hibernate entity annotations in the provided materials; persistence is performed via MyBatis mappers and repository implementations.

7. Persistence and Data Access

Evidence-backed persistence details:

- MyBatis is the primary data access technology (build.gradle includes mybatis-spring-boot-starter and application.properties config:
 - mybatis.configuration.cache-enabled=true
 - mybatis.configuration.default-statement-timeout=3000
 - mybatis.configuration.map-underscore-to-camel-case=true
 - mybatis.configuration.use-generated-keys=true
 - mybatis.type-handlers-package=io.spring.infrastructure.mybatis
 - mybatis.mapper-locations=mapper/*.xml)
- Repository interfaces under io.spring.core.* (ArticleRepository, CommentRepository, ArticleFavoriteRepository, UserRepository) with concrete MyBatis-backed implementations in io.spring.infrastructure.repository (MyBatisArticleRepository,

MyBatisCommentRepository, MyBatisArticleFavoriteRepository, MyBatisUserRepository partial).

- DataSource configuration: SQLite file database (spring.datasource.url=jdbc:sqlite:dev.db; driver org.sqlite.JDBC).
- Transaction management is enabled via @EnableTransactionManagement (MyBatisConfig) and @Transactional used in MyBatisArticleRepository.save method.
- Flyway is present as a dependency (org.flywaydb:flyway-core). No explicit migration files or evidence of migration scripts were found in the provided materials.

8. Security Architecture

Explicit security configuration and behavior from code:

- WebSecurityConfig (extends WebSecurityConfigurerAdapter) defines:
 - JwtTokenFilter bean and registration of the filter before UsernamePasswordAuthenticationFilter.
 - PasswordEncoder bean using BCryptPasswordEncoder.
 - CSRF disabled and CORS configured via corsConfigurationSource bean (allowed origins configured to "*", allowed methods specified, allowed headers specified).
 - Stateless session management (SessionCreationPolicy.STATELESS).
 - Exception handling configured to return HTTP 401 for unauthenticated requests.
 - Authorization rules for endpoints:
 - OPTIONS requests permitted.
 - /graphql and /graphql permitted.
 - GET /articles/feed requires authentication.
 - POST /users and /users/login permitted (public).
 - GET /articles/, /profiles/, /tags permitted (public).
 - All other requests require authentication.
- Controllers use @AuthenticationPrincipal to receive the authenticated User object where applicable.
- JWT configuration properties exist in application.properties:
 - jwt.secret (a secret string present in repository)
 - jwt.sessionTime (value 86400)

9. External Integrations and Communication Points

- No explicit external integrations were detected in the analyzed source code (no HTTP clients, external API client classes, or outbound connectors were found).
- GraphQL support is present via the com.netflix.graphql.dgs dependency and security config allowing /graphql and /graphql; however, explicit GraphQL schema or data fetcher source files were not shown in the provided excerpts.
- Flyway is present as a library dependency for database migrations, but no migration script files were found in the provided materials.

10. Configuration and Runtime Considerations

Configuration artifacts and runtime hints present in the project:

- application.properties contains:
 - JDBC datasource for SQLite dev.db with driver class org.sqlite.JDBC.
 - jackson.deserialization.UNWRAP_ROOT_VALUE=true.
 - mybatis.* configuration entries (mapper locations, type handlers package, cache and timeouts).
 - jwt.secret and jwt.sessionTime properties.

- `image.default` property used by application code (`image.default=https://static.productionready.io/images/smiley-cyrus.jpg`).
- `logging.level.*` settings for specific packages.
- Gradle build: `build.gradle` configures Java 11 compatibility, uses Spring Boot Gradle plugin and DGS codegen plugin; the `generateJava` task is configured with `schemaPaths` and `packageName` for generated GraphQL sources.
- The clean Gradle task deletes `./dev.db` as a pre-clean action (development SQLite file).
- There is no Dockerfile, `docker-compose.yml`, or other containerization descriptor in the repository.
- CI workflow: GitHub Actions workflow is configured to run Gradle clean test on push and pull requests.

11. Observability, Testing and Delivery Evidence

Testing and delivery artifacts:

- Test structure:
 - `src/test/java` present with ~23 test classes.
 - Tests use JUnit 5, Mockito, Spring Boot testing facilities and MockMvc.
 - Tests cover API controllers, application services, repository implementations and infrastructure-level integration tests (e.g., `MyBatisArticleRepositoryTest`).
 - Test helper classes and DB test base classes exist (`DbTestBase`, `TestHelper`).
- CI:
 - `.github/workflows/gradle.yml` executes Gradle clean test on ubuntu-latest and sets up JDK 11.
- Logging:
 - `application.properties` sets DEBUG logging for specific MyBatis-related packages.
- No explicit metrics, tracing, monitoring exporters, or observability agents were detected in the provided materials.

12. Architectural Patterns and Implementation Characteristics

Patterns and implementation evidence:

- Layered architecture: clear separation into controllers (`io.spring.api`), application services (`io.spring.application`), domain/repository interfaces (`io.spring.core`), and infrastructure implementations (`io.spring.infrastructure`).
- Repository pattern: repository interfaces with MyBatis-based implementations (`MyBatisArticleRepository`, `MyBatisCommentRepository`, `MyBatisArticleFavoriteRepository`).
- Global exception handling: `CustomizeExceptionHandler` implements centralized handling for validation and authentication exceptions.
- DTO/use of data transfer objects: application and API layers reference data holder types (`ArticleData`, `UserWithToken`) for response payloads; source files for some DTOs were not detected in the provided materials.
- MyBatis mapper-based data access: mapper XMLs are expected under `mapper/*.xml` (configured), and mapper classes are invoked from infrastructure repository implementations.
- Security pattern: JWT-based stateless authentication filter combined with BCrypt password encoder.

13. Risks, Constraints and Technical Notes

Observed risks, constraints and notable technical items supported by evidence:

- Embedded / file-based database during development:
 - The application is configured to use SQLite (dev.db). This is a single-file database and may limit concurrent write throughput and deployment expectations; evidence shows dev.db is deleted by the clean Gradle task.
- JWT secret stored in application.properties:
 - jwt.secret value is present in repository properties; storing sensitive secrets in repository files is a security risk.
- Missing evidence for migration scripts:
 - Flyway is a declared dependency, but no explicit migration SQL or migration resource files were found in the provided materials.
- Mapper and generated artifacts location expectations:
 - MyBatis mapper XMLs are referenced via mybatis.mapper-locations=mapper/*.xml; absence of mapper XMLs in the provided excerpts may impact build /runtime unless present in the full repository.
- GraphQL schema and code generation:
 - build.gradle configures DGS codegen and a generateJava task with schemaPaths set to src/main/resources/schema, but no schema files were presented in the analyzed artifacts.
- No containerization descriptors:
 - No Dockerfile or compose files were detected; deployment packaging and containerization are not evidenced.
- CORS configuration:
 - CORS is configured with allowed origins set to "*", and allowCredentials set to false. This configuration may be appropriate for some deployments but should be reviewed for production security posture.

14. Recommendations

The following are recommendations only (not implemented features) derived from observed evidence:

- Secrets management:
 - Move jwt.secret (and any other sensitive config) out of version-controlled application.properties into environment variables or a secrets vault compatible with the deployment environment.
- Database and migration:
 - Ensure Flyway migration scripts exist in the repository and are applied at startup or in CI to maintain schema consistency across environments.
 - If production deployment is intended, evaluate replacing SQLite with a production-grade RDBMS or provide clear documentation on operational constraints for SQLite.
- Containerization and deployment:
 - Add a Dockerfile and/or compose manifests if containerized deployment is planned; include CI steps to build and publish artifacts.
- Observability:
 - Add application-level metrics (Prometheus / Micrometer) and structured logging to improve production observability.
- CORS and security posture:
 - Revisit CORS origin settings (avoid wildcard origins for production) and ensure appropriate allowCredentials and header configurations.
- GraphQL artifacts:

- Verify presence and completeness of GraphQL schema files (src/main/resources/schema) if GraphQL endpoints are required; ensure code generation artifacts are committed or generated as part of CI.
- Sensitive logging:
 - Review logging configuration to ensure sensitive information is not logged, and tune log levels for production.

15. Conclusion

The analyzed project is a Spring Boot (Java 11) server application with a layered architecture (controllers, application services, domain interfaces and MyBatis-based infrastructure). It uses Spring Security with JWT for stateless authentication, MyBatis for data access against a configured SQLite datasource, and includes test coverage and a GitHub Actions CI workflow that runs the test suite. Several implementation artifacts are present (custom Jackson module, MyBatis repository implementations, global exception handler). Notable gaps or items requiring attention (based on available evidence) include storage of JWT secret in properties, missing explicit evidence of Flyway migration scripts and GraphQL schema files, and absence of containerization configuration. Recommendations are provided to address configuration, security and operational readiness.